

Indian Beans Payments API — Korean Integration Guide

Accept KakaoPay payments on your Korean site · API host: <https://indianbeans.com>

Two integration options

Pick whichever matches your business flow. The API is the same for both; one request field (`qr_mode`) switches the UX.

Option	Use when	Customer experience
B · QR	Walk-in / POS, retail counter, invoice sent to a client.	Merchant shows one QR. Customer scans with phone camera → KakaoPay app opens pre-filled → tap Pay. No email / no form.
A · <code><iframe></code>	Customer buys from your Korean website / web app.	KakaoPay flow runs inside an <code><iframe></code> on your page. Your parent window receives a <code>postMessage</code> on success.

Prerequisites (both options)

1. An Indian Beans account at <https://indianbeans.com/signup>.
2. An API key: **Portal** → **Settings** → **API Keys** → **Create new key**. Copy it immediately — the full value is shown once.
3. A webhook endpoint on your server (optional but recommended) that we POST to on payment success / failure.

Option B QR walk-in flow

You call the API once per customer and get back a QR code. Show the QR to the walk-in customer; they scan it with their phone camera, KakaoPay opens with the amount pre-filled, they tap Pay. The customer never fills out a form.

Step B1 · Create the payment (your server)

```
curl -X POST https://indianbeans.com/api/public/v1/payments \
-H "Authorization: Bearer YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "amount": 100000,
  "currency": "KRW",
  "payment_methods": ["kakao_pay"],
  "qr": true,
  "qr_mode": "auto_kakao",
  "customer": { "email": "pos@your-site.co.kr" },
  "description": "Order #YOUR_ORDER_ID",
  "webhook_url": "https://your-site.co.kr/webhooks/hub-paid",
  "metadata": { "order_id": "YOUR_ORDER_ID" }
}'
```

Response (201 Created):

```
{
  "id": "cmg...",
  "status": "created",
  "amount": 100000,
  "currency": "krw",
  "qr_mode": "auto_kakao",
  "checkout_url": "https://indianbeans.com/l/AbCdEfGh?auto=kakao",
  "qr_data_url": "...",
  "link_slug": "AbCdEfGh",
}
```

```
"created_at": "2026-10-09T12:34:56Z"
}
```

Step B2 · Display the QR to the customer

The `qr_data_url` is a ready-to-render PNG. Drop it into your POS screen, invoice email, or printed receipt:

```

```

What happens when the customer scans:

1. Customer opens their phone camera and points at the QR.
2. Phone detects the link and offers to open it — customer taps.
3. Phone browser shows a brief *"Opening KakaoPay..."* splash (~1 second).
4. KakaoPay app opens with the amount, merchant and reference pre-filled.
5. Customer taps **Pay** inside KakaoPay. Receipt is shown in the KakaoPay app.

Step B3 · Receive the webhook on your server (recommended)

On payment success/failure we POST your `webhook_url` with a JSON body. Verify, mark the order paid, respond `200 OK`.

```
POST https://your-site.co.kr/webhooks/hub-paid
Content-Type: application/json

{
  "event": "payment.succeeded",
  "payment": { "id": "cmg...", "amount": 100000, "currency": "krw",
               "status": "paid", "method": "kakao_pay" },
  "metadata": { "order_id": "YOUR_ORDER_ID" }
}
```

Why email isn't required: walk-in customers don't supply one. KakaoPay sends its own receipt in-app, so we only need an email for the server-side invoice record. The one in `customer.email` above should be a merchant-side address (e.g. POS, accounting) — never a fabricated customer email.

Option A Iframe embed flow

For when the customer is already on your Korean website buying something. The KakaoPay checkout renders inside an `<iframe>` you drop on your page; your parent window gets a `postMessage` on success.

Step A1 · Create the payment (your server)

```
curl -X POST https://indianbeans.com/api/public/v1/payments \
-H "Authorization: Bearer YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "amount": 100000,
  "currency": "KRW",
  "payment_methods": ["kakao_pay"],
  "customer": {
    "email": "buyer@your-site.co.kr",
    "name": "홍길동"
  },
  "description": "Order #YOUR_ORDER_ID",
  "webhook_url": "https://your-site.co.kr/webhooks/hub-paid",
  "metadata": { "order_id": "YOUR_ORDER_ID" }
}'
```

Note: no `qr_mode` field — that's what switches to Option B. For the iframe flow omit it.

Response includes an iframe-ready URL:

```
{
  "id": "cmg...",
  "checkout_url": "https://indianbeans.com/l/AbCdEfGh",
  "embed_url": "https://indianbeans.com/l/AbCdEfGh?embed=1",
  "link_slug": "AbCdEfGh"
}
```

Step A2 · Embed on your customer-facing page

```
<iframe
  id="hub-payment"
  src="EMBED_URL"
  width="100%"
  height="700"
  frameborder="0"
  allow="payment"
  style="border:0; max-width:480px; display:block; margin:0 auto;"
></iframe>

<script>
  window.addEventListener('message', function(e) {
    if (e.origin !== 'https://indianbeans.com') return; // always verify origin
    if (e.data && e.data.type === 'payment:succeeded') {
      document.getElementById('hub-payment').style.display = 'none';
      document.getElementById('thank-you').style.display = 'block';
      fetch('/mark-paid?invoice=' + encodeURIComponent(e.data.invoiceId));
    }
  });
</script>
```

Step A3 · Receive the webhook

Same shape as Option B. See **Step B3** on the previous page.

Why both `postMessage` + `webhook`? The `postMessage` is instant UI feedback for the customer in the iframe. The webhook is the authoritative server-to-server confirmation — always mark orders paid from the webhook, never only from the browser message.

Reference

Request fields — POST `/api/public/v1/payments`

Field	Type	Notes
<code>amount</code>	integer	Smallest unit (₩ for KRW — pass whole won; cents for USD). Required.
<code>currency</code>	string	ISO 4217 code. KRW, USD, CAD, EUR, etc.
<code>payment_methods</code>	string[]	<code>["kakao_pay"]</code> . (Also supports <code>["paypal"]</code> , <code>["alipay"]</code> .)
<code>qr</code>	boolean	<code>true</code> → <code>qr_data_url</code> included in response.
<code>qr_mode</code>	string	<code>"auto_kakao"</code> → Option B (QR walk-in). Omit for Option A (iframe).
<code>customer.email</code>	string	Required. For Option B this is a merchant-side address — not shown to the customer.
<code>customer.name</code>	string	Optional. Supports Korean characters.
<code>description</code>	string	≤ 500 chars. Shown on the Paddle hosted checkout page.
<code>webhook_url</code>	string	HTTPS. Where we POST on payment events.
<code>metadata</code>	object	Any JSON. Echoed in the webhook payload.

Response fields

Field	Appears for	Use for
<code>id</code>	Both	Internal reference (an invoice or link id).
<code>checkout_url</code>	Both	Full-page checkout. Share via email, SMS, or print. For Option B this includes <code>?auto=kakao</code> .
<code>embed_url</code>	Option A	iframe <code>src</code> . Chrome stripped, <code>postMessage</code> on paid.
<code>qr_data_url</code>	Both (when <code>qr: true</code>)	Inline PNG QR of <code>checkout_url</code> . Scan with phone camera.
<code>qr_mode</code>	Option B only	Echoes <code>"auto_kakao"</code> so you can confirm the mode.
<code>link_slug</code>	Both	The short slug portion of <code>checkout_url</code> .

postMessage event (Option A only — iframe → parent)

```
{
  "type":      "payment:succeeded",
  "invoiceId": "cmg...",
  "invoiceNumber": "INV-2026-0042",
  "amountCents": 100000,
  "currency":  "KRW"
}
```

Webhook body (both options)

```
POST https://your-site.co.kr/webhooks/hub-paid
Content-Type: application/json

{
  "event":      "payment.succeeded" | "payment.failed",
  "payment": {
    "id":        "cmg...",
    "amount":    100000,
    "currency":  "krw",
    "status":    "paid" | "failed",
    "method":    "kakao_pay"
  },
  "customer": { "email": "...", "name": "..." },
  "metadata": { "order_id": "YOUR_ORDER_ID" }
}
```

Testing checklist

Option B — QR flow

- POST the request. Response **201** with `qr_data_url` and `qr_mode: "auto_kakao"`.
- Open `checkout_url` in a desktop browser — “Opening KakaoPay...” splash renders, Paddle checkout follows.
- Save `qr_data_url` as a PNG (decode the base64). Display it on screen or print it.
- Scan with a Korean phone camera that has the KakaoPay app installed. KakaoPay should open with the payment pre-filled.
- Complete a test payment (KRW minimum ≈ ₩1,000). Confirm your webhook endpoint received the `payment.succeeded` POST.

Option A — iframe flow

- POST the request. Response **201** with `embed_url`.
- Open `embed_url` directly in a browser — bare-chrome checkout loads.
- Drop the iframe snippet onto a test page on your site. Attach the `message` listener.
- Complete a test payment with a real Korean phone + KakaoPay. Confirm your parent-window listener fires and your webhook receives the event.

Minimum amounts & limits

- **KRW minimum per transaction:** ~₩1,000. Smaller amounts are rejected by Paddle with a “transaction balance less than minimum” error.
- Amounts are integers in the currency's smallest unit. For KRW that's whole won (no decimals). For USD / CAD / EUR that's cents.
- One payment request = one single-use session. Create a new one per order.

Security: never ship your API key in frontend code. Always call `/api/public/v1/payments` from your backend. The returned `checkout_url` / `embed_url` / `qr_data_url` are safe to send to the browser — each is a short-lived single-use session.

Support

Technical questions: hello@indianbeans.com · Response within one business day. Include your API key prefix (first 10 chars) and the `id` from the response.